MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #3 – Introduction to the MiniBot

Student Names: Brady Cabral and Emily Vargas-Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: March 4, 2026

Summary:
This lab introduced the operation and programming of the MiniBot robot using an ItsyBitsy microcontroller. The objective of this lab was to become familiar with the robot's hardware and software, calibrate the motor control parameters, and program the robot to go forward and return to its starting position. Activities included: a) using a digital multimeter (DMM) op for voltage measurements, b) adjusting motor parameters and c) tuning the robot's motion to minimize drift.

Hardware:
The setup for the MiniBot is shown in **Fig 1.** Components on the Minibot include LEDs, an ItsyBitsy, ServoMotors, buttons, resistors, battery and an assortment of other sensors and motors which weren't used in this lab. The testing setup is shown in **Fig 2.** Which shows the MiniBot placed on the floor with a ruler to record the distance the bot moves before turning around to return to origin.

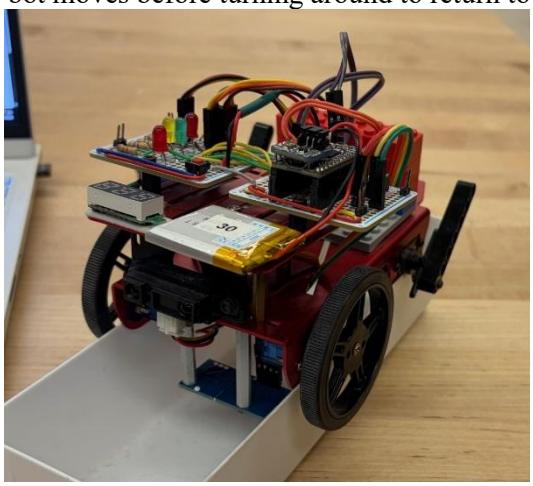

Fig. 1. MiniBot on plastic stand on table

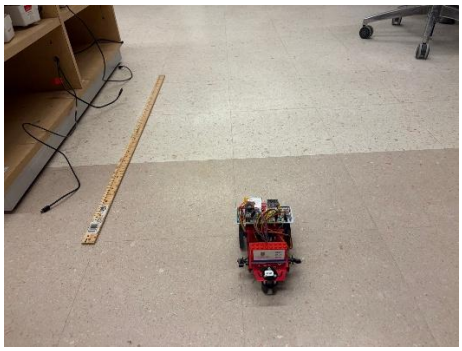

Fig. 2. MiniBot in testing setup on floor with ruler

Software:
A listing of the program ***Group30Lab3MiniBotFwdReturn*** used in the lab is given in **Appendix A**. The program primarily instructs the robot to move forward around 70cm, pause for two seconds then turn 180 degrees then move forward back to start. The red LED is turned on when moving forward, the yellow LED when pausing and the green LED is turned on when turning. The other LEDs are turned off when another is turned on.

Results*:*
The relationship between delta variable for the motor and distance travelled in four seconds is shown in **Table 1**. The results of adjusting variables such as offset, turning time, forward and reverse time showing how it affects performance is in **Table 2**. **Table 3** shows the loaded and unloaded voltage of the MiniBot when testing. Answers to questions given in the lab are provided below.

Q1. Yes, it would have been possible to correct drift by increasing the speed of the opposite wheel as opposed to slowing down the drifting wheel. However, in practice, the slowing wheel approach is better because it tends to be more consistent with the battery voltage drops (by draining less power from the battery).

Q2. The maximum current of a Feetech motor driver is 1.3A, 650 mA for the FM90 motor and 40mA for an Arduino. This is significant because the Arduino pins cannot power motors (as the current of motors exceeds the maximum current of an Arduino pin). As an interface between a low power controller (like and Arduino) and a high power motor, a motor driver must be used to regulate speed, direction, and torque. (Sources: https://akizukidenshi.com/goodsaffix/FT-SMC-2CH.pdf?utm_, https://abra-electronics.com/electromechanical/motors/dc-motors/fm90-dc-motor-in-micro-servo-body.html?utm_, https://store-usa.arduino.cc/products/arduino-uno-wifi-rev2?srsltid=AfmBOorOz0DnuYWzgB9TnL4dZIqNTX1vOXiBpKFMw9Gc8in-j5o5qJhj&utm_, https://orbray.com/magazine_en/archives/3659)

Q3. The operating voltage range for an ItsyBitsy is 3.5-6V, 4-9 V for the Feetech motor driver, and 4.8-6V for the FM90 motor. This is significant because one must ensure that the battery/rail voltage stays inside each component's range for appropriate functioning. (Sources: https://learn.adafruit.com/introducting-itsy-bitsy-32u4/overview, https://akizukidenshi.com/goodsaffix/FT-SMC-2CH.pdf?utm_ , https://abra-electronics.com/electromechanical/motors/dc-motors/fm90-dc-motor-in-micro-servo-body.html?utm_,)

All figures and tables need to be numbered and cited in the text.

Conclusions:
This lab provided practice experience with the MiniBot and its motor control mechanism. One important concept learned was how small differences in motor control parameters can cause significant drift in the robot's trajectory. A notable variation between trials was also recorded, even when not changing any parameters (namely because of battery drainage, floor friction and slight difference in starting orientation).

Table 1. Table showing how distance travelled after 4 seconds changes with delta **Lab #3**

	Distance Travelled in 4 seconds (m)			
Delta	1st Trial	2nd Trial	3rd Trial	avg
20	1.00	1.05	1.05	1.03
25	1.05			
30	1.10			

35	1.05			

Table 2. Trial and error tuning of the Minibot shown in **Lab #3**

Trial	Battery (V)	Forward Time	Forward Offset	Dist (cm)	Drift (cm)	Turn time	Angle (deg)	Return Time	Return Offset	Overshoot	Drift
1	N/A	3	0	75	20	1000	100	3	0		
2	3.82	3	0	60	25	1500	170	3	0		
3	3.75	3	0	65	20	1650	180	3	0		
4	3.76	3	2	75	2	1650	210	3	2		
5	3.73	3	2	75	9	1500	180	3	2		1
6	3.75	3	3	75	2	1500	200	3	3		10
7	3.73	3	3	75	2	1400	190	3	3		5
8	3.71	3	3	75	2	1380	183	3	3	4	4

Table 3. Initial loaded and unloaded voltages of the Minibot in **Lab #3**

Item	Voltage (V)
Unloaded voltage	4.16
Loaded voltage	3.87

Appendix A - Program Listing:

```
/*********************************************************
  M103Lab3MiniBotFwdReturn
  B.Cabral & E. Vargas-Penaranda, 04/03/2025

 This program is used to test Minibot
 Robot goes forward then turn 180 degrees then goes
forward back to start
*********************************************************/
#include <Servo.h>
Servo leftWheel;
Servo rightWheel;

// Pin Assignments
int RED = 10;          // red LED Pin
int GRN = 9;           // green LED Pin
int YLW = 5;           // yellow LED Pin
int BUTTON = 7;        // pushbutton Pin
int MOTOR_R = 3;       // right motor signal pin
int MOTOR_L = 4;       // left motor signal pin

// Delta = speed above (+) or below (-) stop speed
(must be positive)
const int stopPulse = 149;  // stop speed for motors
(default = 155)
const int delta = 20;       // pulse differential (default =
20)
const int offset = 3;       // offset, slows right wheel
(default = 0)

// Set-up Routine
void setup() {

// Initialize LED pins as outputs.
  pinMode(GRN, OUTPUT);
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);

// Initialize button pins as inputs
  pinMode(BUTTON, INPUT);

//initialize motor control pins as servo pins
  leftWheel.attach(MOTOR_L);
  rightWheel.attach(MOTOR_R);

// Initialize serial printout
  Serial.begin(9600);   // default 9600

}

// Main Routine
void loop() {

// wait for button press
   do {
    runMotors(0,0);
    toggleLED(GRN);        //motors stopped, Green LED
flashing
   } while(digitalRead(BUTTON)== LOW);
    delay(200); // time to clear button

// move robot forward
     turnOnLED(RED);
     runMotors(delta,delta-offset);
     delay(3000);
// pause
     turnOnLED(YLW);
     runMotors(0,0);
     delay(2000);

//Turn Robot around
     turnOnLED(GRN);
     runMotors(delta,0);
     delay(1380);

// move robot forward
     turnOnLED(RED);
     runMotors(delta,delta-offset);
     delay(3000);
}

//********** Functions (subroutines) ******************

// run robot wheels
void runMotors(int deltaL, int deltaR)
{
  int pulseL = (stopPulse + deltaL)*10;    //length of
pulse in microseconds
  int pulseR = (stopPulse + deltaR)*10;
  leftWheel.writeMicroseconds(pulseL);
  rightWheel.writeMicroseconds(pulseR);
}

// Turn on a single LED and turn others off
void turnOnLED(int colour)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(colour, HIGH);
}

//Toggle an LED on/off
void toggleLED(int colour){
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(colour, HIGH);
```

```
  delay(250);
  digitalWrite(colour, LOW);
  delay(250);
}
```